

1. Equivalent Forms of the Fused Stencil Filter

The fused stencil filter can be expressed in several equivalent notations. Each form emphasizes a different aspect of the computation and connects to different implementation strategies and bodies of literature.

1.1. Summation Form

The original per-pixel formulation iterates over all unique stencil positions for a given orientation.

$$R_k = \sum_{\ell=1}^{N'_k} \alpha_{k,\ell} \cdot f(X_0 + \tilde{\delta}_\ell^x, Y_0 + \tilde{\delta}_\ell^y), \quad k^* = \arg \max_k |R_k|$$

This form makes the structure of the computation explicit. Each $\alpha_{k,\ell}$ is a precomputed scalar weight encoding polynomial fitting, pseudoinverse extraction, and Gaussian line weighting. The summation runs over N'_k unique pixel positions in the deduplicated stencil for orientation k .

1.2. Inner Product Form

Collecting the weights and sampled intensities into vectors yields a compact algebraic expression.

$$R_k = \langle \alpha_k, \mathbf{f}_k \rangle$$

Here $\alpha_k \in \mathbb{R}^{N'_k}$ is the vector of fused weights for orientation k , and $\mathbf{f}_k \in \mathbb{R}^{N'_k}$ is the vector of image intensities sampled at the corresponding stencil positions. The angle brackets denote the Euclidean inner product. This form emphasizes that the filter response is a single dot product per orientation per pixel.

1.3. Matrix Form

Evaluating all N_s orientations simultaneously can be written as a single matrix–vector product.

$$\mathbf{R} = \mathbf{A} \mathbf{f}$$

The matrix $\mathbf{A} \in \mathbb{R}^{N_s \times N}$ has one row per candidate orientation, where N is the total number of unique pixel positions across all stencils. The vector $\mathbf{f} \in \mathbb{R}^N$ contains the image intensities at those positions. Each row of $\mathbf{A}$ is a zero-padded version of $\alpha_k^\top$, with nonzero entries only at the columns corresponding to that orientation’s stencil positions. The winning orientation is then $k^* = \arg \max_k |R_k|$ applied to the entries of $\mathbf{R}$.

1.4. Convolution Form

Embedding the fused weights into a 2D kernel converts the filter into a standard spatial convolution.

$$R_k(X_0, Y_0) = (h_k \star f)(X_0, Y_0)$$

The kernel h_k is a sparse two-dimensional array with the value $\alpha_{k,\ell}$ placed at offset $(\tilde{\delta}_\ell^x, \tilde{\delta}_\ell^y)$ and zeros elsewhere. The operator $\star$ denotes cross-correlation. This form maps directly to GPU primitives such as `conv2d` and makes clear that the fused stencil is a standard convolutional filter, compatible with existing deep learning and image processing frameworks without modification.

1.5. Tensor Contraction Form

Processing the full image across all orientations simultaneously can be written as a batched convolution producing a three-dimensional response volume.

$$G = \mathcal{H} \star f$$

The filter bank $\mathcal{H} \in \mathbb{R}^{N_s \times H \times W}$ stacks all N_s oriented kernels h_k along a leading dimension, where $H \times W$ is the spatial extent of the largest stencil. The output $G \in \mathbb{R}^{N_s \times N_h \times N_v}$ is a response volume indexed by orientation and spatial position. The per-pixel gradient magnitude and edge angle are extracted pointwise as follows.

$$k^*(X_0, Y_0) = \arg \max_k |G_{k, X_0, Y_0}|$$

$$\text{magnitude} = |G_{k^*, X_0, Y_0}|, \quad \text{angle} = \theta_{k^*}$$

This form is the natural representation for a fully parallelized GPU implementation, where the entire filter bank is applied as a single batched convolution and the orientation selection reduces to a pointwise argmax over the leading axis.